

NEURAL NETWORK DEEP LEARNING

ICP 7

700758238

AKHIL KASANAGOTTU

GitHub:

Repository URL for the source code:

https://github.com/axk82380/NNPL/tree/main/ICP_7

Video Link:

https://drive.google.com/file/d/1kRO_f3or7OJLJHqYYD0lvV60jasDbCXW/view?usp=sharing

1. Save the model and use the saved model to predict on new text data (ex, “A lot of good things are happening. We are respected again throughout the world, and that's a great thing.@realDonaldTrump”)

```
In [3]: import pandas as pd #Basic packages for creating dataframes and loading dataset
import numpy as np

import matplotlib.pyplot as plt #Package for visualization

import re #importing package for Regular expression operations

from sklearn.model_selection import train_test_split #Package for splitting the data

from sklearn.preprocessing import LabelEncoder #Package for conversion of categorical to Numerical

from keras.preprocessing.text import Tokenizer #Tokenization
from tensorflow.keras.preprocessing.sequence import pad_sequences #Add zeros or crop based on the length
from keras.models import Sequential #Sequential Neural Network
from keras.layers import Dense, Embedding, LSTM, SpatialDropout1D #For Layers in Neural Network
from keras.utils.np_utils import to_categorical
```

```
In [4]: df = pd.read_csv("D:/UCM/1st Semester/Neural Network & Deep Learning/ICP10/Sentiment.csv")
```

```
In [27]: import pandas as pd

# Load the dataset as a Pandas DataFrame
dataset = pd.read_csv(path_to_csv, header=0)

# Select only the necessary columns 'text' and 'sentiment'
mask = dataset.columns.isin(['text', 'sentiment'])
data = dataset.loc[:, mask]

# Keeping only the necessary columns
```

Importing the packages, reading the .csv file, loading the dataset as a Panda Data Frame, selecting only the necessary columns that are 'text' and 'sentiment'.

```

In [29]: data['text'] = data['text'].apply(lambda x: x.lower())
data['text'] = data['text'].apply((lambda x: re.sub('[^a-zA-Z0-9\s]', '', x)))

<ipython-input-29-cee1da567eb8>:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
data['text'] = data['text'].apply(lambda x: x.lower())
<ipython-input-29-cee1da567eb8>:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
data['text'] = data['text'].apply((lambda x: re.sub('[^a-zA-Z0-9\s]', '', x)))

In [30]: for idx, row in data.iterrows():
row[0] = row[0].replace('rt', ' ') #Removing Retweets

```

Using lambda function to convert to lower case.

Using re.sub() regular expression to replace any non-alphanumeric characters and non-whitespace characters with an empty string.

Then, Removing the retweets.

```

In [31]: max_features = 2000
tokenizer = Tokenizer(num_words=max_features, split=' ') #Maximum words is 2000 to tokenize sentence
tokenizer.fit_on_texts(data['text'].values)
X = tokenizer.texts_to_sequences(data['text'].values) #taking values to feature matrix

In [32]: X = pad_sequences(X) #Padding the feature matrix

embed_dim = 128 #Dimension of the Embedded Layer
lstm_out = 196 #Long short-term memory (LSTM) Layer neurons

```

Total number of words to tokenize sentence is limited to 2000 and taking values to feature matrix. Now padding the feature matrix with Dimension 128 and long short-term memory to 196.

```

In [33]: def createmodel():
model = Sequential() #Sequential Neural Network
model.add(Embedding(max_features, embed_dim, input_length = X.shape[1])) #input dimension 2000 Neurons, output dimension 128 Neurons
model.add(LSTM(lstm_out, dropout=0.2, recurrent_dropout=0.2)) #Drop out 20%, 196 output Neurons, recurrent dropout 20%
model.add(Dense(3, activation='softmax')) #3 output neurons[positive, Neutral, Negative], softmax as activation
model.compile(loss = 'categorical_crossentropy', optimizer='adam', metrics = ['accuracy']) #Compiling the model
return model
# print(model.summary())

```

the 'createmodel()' function creates a Sequential model with an Embedding layer to handle the input data, an LSTM layer to extract features from the input sequences, and a Dense layer with softmax activation to predict the sentiment class. The model is then compiled and returned.

```
In [34]: lencoder = LabelEncoder() #Applying Label Encoding on the Label matrix
ger_encoded = lencoder.fit_transform(data['sentiment']) #fitting the model
to_categorical(integer_encoded)
ain, X_test, Y_train, Y_test = train_test_split(X,y, test_size = 0.33, random_state = 42) #67% training data, 33% test data split
```

the 'fit_transform()' method of the 'LabelEncoder' instance is to convert the categorical sentiment labels in the data DataFrame to numerical values. The resulting numerical values are stored in the integer_encoded variable. Then this splits the input data (X) and target data (y) into training and testing sets using the train_test_split function from the Scikit-learn model selection module. The testing set is set to be 33% of the total data (test_size=0.33) and the random seed is set to 42 (random_state=42). The resulting variables are X_train (training input data), X_test (testing input data), Y_train (training target data), and Y_test (testing target data).

```
In [36]: print(model.metrics_names) #metrics of the model
['loss', 'accuracy']
```

#1. Save the model and use the saved model to predict on new text data (ex, "A lot of good things are happening. We are respected again throughout the world, and that's a great thing.@realDonaldTrump")

```
In [37]: model.save('sentimentAnalysis.h5') #Saving the model
```

```
In [38]: from keras.models import load_model #Importing the package for importing the saved model
model = load_model('sentimentAnalysis.h5') #Loading the saved model
```

WARNING:tensorflow:Layer lstm will not use cuDNN kernels since it doesn't meet the criteria. It will use a generic GPU kernel as a fallback when running on GPU.

```
In [39]: print(integer_encoded)
print(data['sentiment'])

[1 2 1 ... 2 0 2]
0      Neutral
1      Positive
2      Neutral
3      Positive
4      Positive
...
13866 Negative
13867 Positive
13868 Positive
13869 Negative
13870 Positive
Name: sentiment, Length: 13871, dtype: object
```

Now printing the metrics, saving the model, then printing the accuracy of each data.

```
In [44]: # Predicting on the text data
sentence = ['A lot of good things are happening. We are respected again throughout the world, and that is a great thing.@realDonaldTrump']
sentence = tokenizer.texts_to_sequences(sentence) # Tokenizing the sentence
sentence = pad_sequences(sentence, maxlen=28, dtype='int32', value=0) # Padding the sentence
sentiment_probs = model.predict(sentence, batch_size=1, verbose=2)[0] # Predicting the sentence text
sentiment = np.argmax(sentiment_probs)

print(sentiment_probs)
if sentiment == 0:
    print("Neutral")
elif sentiment < 0:
    print("Negative")
elif sentiment > 0:
    print("Positive")
else:
    print("Cannot be determined")

1/1 - 0s - 22ms/epoch - 22ms/step
[0.3347626 0.16386913 0.5013683 ]
Positive
```

Now predicting the model by testing on a sentence. And the result is positive.

2. Apply GridSearchCV on the source code provided in the class

```
In [45]: from keras.wrappers.scikit_learn import KerasClassifier #importing Keras classifier
from sklearn.model_selection import GridSearchCV #importing Grid search CV

model = KerasClassifier(build_fn=createmodel,verbose=2) #initiating model to test performance by applying multiple hyper parameters
batch_size= [10, 20, 40] #hyper parameter batch_size
epochs = [1, 2] #hyper parameter no. of epochs
param_grid= {'batch_size':batch_size, 'epochs':epochs} #creating dictionary for batch size, no. of epochs
grid = GridSearchCV(estimator=model, param_grid=param_grid) #Applying dictionary with hyper parameters
grid_result= grid.fit(X_train,Y_train) #Fitting the model
# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_)) #best score, best hyper parameters

186/186 - 26s - loss: 0.6874 - accuracy: 0.7052 - 26s/epoch - 139ms/step
47/47 - 1s - loss: 0.7363 - accuracy: 0.6889 - 748ms/epoch - 16ms/step

WARNING:tensorflow:Layer lstm_30 will not use cuDNN kernels since it doesn't meet the criteria. It will use a generic GPU kernel as fallback when running on GPU.

Epoch 1/2
186/186 - 37s - loss: 0.8370 - accuracy: 0.6371 - 37s/epoch - 198ms/step
Epoch 2/2
186/186 - 26s - loss: 0.6795 - accuracy: 0.7098 - 26s/epoch - 140ms/step
47/47 - 1s - loss: 0.7777 - accuracy: 0.6652 - 730ms/epoch - 16ms/step

WARNING:tensorflow:Layer lstm_31 will not use cuDNN kernels since it doesn't meet the criteria. It will use a generic GPU kernel as fallback when running on GPU.

Epoch 1/2
465/465 - 74s - loss: 0.8138 - accuracy: 0.6524 - 74s/epoch - 159ms/step
Epoch 2/2
465/465 - 62s - loss: 0.6739 - accuracy: 0.7108 - 62s/epoch - 134ms/step
Best: 0.681371 using {'batch_size': 20, 'epochs': 2}
```

Initiating model to test performance by applying multiple hyper parameters. Defining the batch_size, no. of epochs and then creating dictionary for batch size & no. of epochs. Then fitting the model and summarizing the results. The accuracy is 0.7108 with batch size 20 and epochs as 2.